1. Organización, acceso y personal

Restaurante
- -nombre : String
- -rut : String
+ obtenerRoles() : List<Rol>

«abstract» Usuario
- -email : String
- -passwordHash : String
- -activo : Boolean
+ autenticar(credencial) : Boolean

Empleado
- -nombreCompleto : String
- -fotoUrl : String
- -horasSemanalesMax : Integer
+ puedeAcceder(permiso) : Boolean
+ arquetipo() : Arquetipo
+ obtenerRol() : Rol
+ obtenerSucursales() : List<Sucursal>

CredencialBiometrica
- -credentialId : String
- -clavePublica : String
- -dispositivo : String
- -contadorFirmas : Integer
+ verificarFirma(desafio : String, firma : String) : Boolean

Comensal
- -nombre : String
- -telefono : String

Rol
- -nombre : String
- -predeterminado : Boolean
- -arquetipo : Arquetipo
- -activo : Boolean
+ tienePermiso(permiso) : Boolean
+ obtenerPermisos() : List<Permiso>

Sucursal
- -nombre : String
- -direccion : String
- -horarioApertura : Time
- -horarioCierre : Time
- -toleranciaMinDuenho : Integer
+ buscarMesaDisponible(inicio : DateTime, personas : Integer) : Mesa

Permiso
- -codigo : String
- -modulo : String
- -accion : String
+ listarPorModulo() : Map<String, List<Permiso>>

Turno
- -fecha : Date
- -franja : FranjaServicio
+ horas() : Decimal

HorarioServicio
- -franja : FranjaServicio
- -horaInicio : Time
- -horaFin : Time
+ duracionHoras() : Decimal

trabaja en
menú activo
horas de la franja

2. Salón, reservas, consumo y pago

Mesa
- -numero : Integer
- -capacidad : Integer
- -posicionX : Decimal
- -posicionY : Decimal
- -activa : Boolean
+ estaDisponible(inicio : DateTime) : Boolean

MedioPagoGuardado
- -tokenPasarela : String
- -marca : String
- -ultimos4 : String

Reserva
- -inicio : DateTime
- -cantidadPersonas : Integer
- -modoAsignacion : ModoAsignacion
- -estado : EstadoReserva
+ fin() : DateTime
+ activar() : void
+ cancelar() : void
+ vencer() : void
+ finalizar() : void
+ cambiarEstado(nuevo : EstadoReserva) : void

se abre al activarse

CuentaMesa
- -abiertaEn : DateTime
- -cerradaEn : DateTime
- -estado : EstadoCuenta
+ total() : Decimal
+ saldoPendiente() : Decimal
+ estaAbierta() : Boolean
+ cerrar() : void

Pedido
- -numero : Integer
- -creadoEn : DateTime
- -estado : EstadoPedido
- -motivoCancelacion : String
+ avanzar(Estado) : void
+ cancelar(motivo : String) : void
+ cambiarEstado(nuevo : EstadoPedido) : void

Pago
- -monto : Decimal
- -metodo : MetodoPago
- -estado : EstadoPago
- -fecha : DateTime
- -referenciaExterna : String
+ aprobar() : void
+ rechazar() : void
+ cambiarEstado(nuevo : EstadoPago) : void

LineaPedido
- -cantidad : Integer
- -precioUnitario : Decimal
- -observaciones : String
+ subtotal() : Decimal

BeneficioAplicado
- -montoDescontado : Decimal

Comprobante
- -numero : String
- -fechaEmision : DateTime
- -total : Decimal

publica
publica progreso
vencer()

3. Menú, recetas e inventario

Menu
- -nombre : String
- -descripcion : String
- -vigenciaDesde : Date
- -vigenciaHasta : Date
+ estaVigente(fecha : Date) : Boolean

ItemMenu
- -nombre : String
- -descripcion : String
- -precio : Decimal
- -fotoUrl : String
- -disponible : Boolean
- -activo : Boolean
+ estaAlAlcance(fan) : Boolean
+ hayStockEn(sucursal, cantidad) : Boolean
+ obtenerReceta() : Receta
+ obtenerModeloRA() : ModeloRA

«builder» **RecetaBuilder**
+ desde(receta : Receta) : RecetaBuilder
+ paraItem(item : ItemMenu) : RecetaBuilder
+ paraIntermedio(intermedio : ProductoIntermedio) : RecetaBuilder
+ agregarLinea(componenteInventario, Decimal, UnidadMedida, Decimal) : RecetaBuilder
+ conRendimiento(valor : Decimal) : RecetaBuilder
+ conInstrucciones(texto : String) : RecetaBuilder
+ build() : Receta

Categoria
- -nombre : String
- -requiereReceta : Boolean

Receta
- -rendimiento : Decimal
- -instrucciones : String
+ costoTotal() : Decimal
+ insumosNecesarios(cantidad : Decimal) : Map<Insumo, Decimal>

ModeloRA
- -urlModelo : String
- -urlModeloIOS : String
- -escalaReal : Decimal
- -porcionGramos : Integer

LineaReceta
- -cantidad : Decimal
- -unidad : UnidadMedida
- -porcentajeMermaEsperada : Decimal
+ cantidadNetaEnBase() : Decimal
+ cantidadBrutaEnBase() : Decimal

«create»
valida contiene()
se elabora con
registrado por

«abstract» **ComponenteInventario**
- -nombre : String
- -unidadBase : UnidadMedida
- -activo : Boolean
+ costoUnitario() : Decimal
+ contiene(c : ComponenteInventario) : Boolean

Insumo
- -costoCompra : Decimal
+ costoUnitario() : Decimal
+ actualizarCosto(costoCompra : Decimal) : void

ProductoIntermedio
+ costoUnitario() : Decimal
+ obtenerReceta() : Receta
+ contiene(c : ComponenteInventario) : Boolean

Existencia
- -cantidadActualEnBase : Decimal
+ alcanza(cantidad : Decimal) : Boolean
+ descontar(cantidad : Decimal) : void
+ ingresar(cantidad : Decimal) : void
+ ajustar(cantidadContada : Decimal) : Decimal

[unica por insumo y sucursal]

MovimientoStock
- -tipo : TipoMovimiento
- -cantidadEnBase : Decimal
- -costoUnitario : Decimal
- -fecha : DateTime
- -motivo : String

4. Fidelización y promociones

CuentaFidelidad
- -puntos : Integer
- -cantidadVisitas : Integer
- -montoAcumulado : Decimal
+ acreditar(pago : Pago) : void
+ registrarVisita(cuentaMesa : CuentaMesa) : void
+ canjear(recompensa : Recompensa, pago : Pago) : void
+ itemsElegiblesDisponibles() : List<ItemMenu>
+ progresoHacia(recompensa : Recompensa) : Decimal

GrupoBeneficios
- -nombre : String
- -habilitado : Boolean

ProgramaFidelidad
- -puntosPorUnidadMonetaria : Decimal
- -activo : Boolean
+ calcularPuntos(monto : Decimal) : Integer

registrarVisita()

Visita
- -fecha : DateTime

MovimientoPuntos
- -tipo : TipoMovimientoPuntos
- -puntos : Integer
- -fecha : DateTime

«abstract» **Beneficio**
- -nombre : String
- -tipoDescuento : TipoDescuento
- -valor : Decimal
- -vigenciaDesde : DateTime
- -vigenciaHasta : DateTime
- -stockDisponible : Integer
- -activo : Boolean
+ estaVigente() : Boolean
+ calcularDescuento(monto : Decimal) : Decimal
+ consumirStock() : void
+ aplicaA(cuenta, pago) : Boolean

Promocion

Recompensa

Se persiste en la tabla BENEFICIO como criterio_tipo = criterio_valor (solo Recompensa): el criterio se reconstruye a partir del tipo. desde = último canje de esa recompensa (una vez por ciclo).

«abstract» **CriterioElegibilidad**
+ estElegible(cuenta : CuentaFidelidad, desde : DateTime) : Boolean

CriterioPorMonto
- -montoRequerido : Decimal

CriterioPorVisitas
- -visitasRequeridas : Integer

CriterioPorPuntos
- -puntosRequeridos : Integer

CriterioNuevoCliente
- -numeroVisita : Integer

5. Simulador de apertura y planificación de turnos

PlanificacionTurnos
- -semanaDesde : Date
- -estado : EstadoPlanificacion
- -generadaEn : DateTime
- -confirmadaEn : DateTime
+ confirmar() : void
+ regenerar() : void

ConfiguracionApertura
- -semanasHistoricoMinimo : Integer
- -cubiertosPorMozo : Decimal
- -platosPorCocineroHora : Decimal

se basa en

Pronostico
- -fechaServicio : Date
- -franja : FranjaServicio
- -cubiertosEstimados : Integer
- -limiteInferior : Integer
- -limiteSuperior : Integer
- -ingresoEstimado : Decimal
- -errorMAPE : Decimal
- -metodo : MetodoPronostico
- -generadoEn : DateTime

genera
calcula
crea faltantes
informa
usa
ejecuta
usa
faltante de personal

«service» **AsistenteApertura**
+ progresoDatos(sucursal) : ProgresoDatos
+ generarPronostico(sucursal, fecha) : List<Pronostico>
+ planificarSemana(sucursal, semanaDesde) : PlanificacionTurnos

Alerta
- -tipo : TipoAlerta
- -severidad : Severidad
- -descripcion : String
- -fechaServicio : Date
- -franja : FranjaServicio
- -arquetipo : Arquetipo
- -cantidad : Integer
- -creadaEn : DateTime
- -resuelta : Boolean

DemandaItem
- -cantidadEstimada : Integer

«value object» **DotacionRequerida**
- -fecha : Date
- -franja : FranjaServicio
- -arquetipo : Arquetipo
- -cantidad : Integer

«value object» **ProgresoDatos**
- -semanasDisponibles : Integer
- -semanasRequeridas : Integer
+ alcanzaMinimo() : Boolean
+ porcentaje() : Decimal

«interface» **EstrategiaPronostico**
+ pronosticar(historial, fecha, franja) : Pronostico

«interface» **VerificadorStock**
+ verificar(pronostico) : List<Alerta>

«interface» **EstrategiaAsignacionTurnos**
+ asignar(dotacion, empleados) : List<Turno>

PromedioMovilPonderado
NaiveEstacional
RegresionConEstacionalidad
VerificadorStock
VerificadorMesas
AsignacionGolosaBalanceada

Eventos de dominio y tiempo real (Observer)

«schedule» **TareaVencimientoReservas**
+ ejecutar() : void

«service» **BusEventos**
+ suscribir(tipoEvento, suscriptor) : void
+ publicar(evento) : void

distribuye
estado

«value object» **EventoDominio**
- -tipo : TipoEvento
- -sucursalId : UUID
- -comensalId : UUID
- -ocurridoEn : DateTime
- -datos : JSON

«interface» **Suscriptor**
+ manejar(evento) : void

origen
publica

GeneradorAlertas
NotificadorSalon
NotificadorStaff
NotificadorComensal
RegistroVisitas

«service» **CanalTiempoReal**
+ emitir(canal, mensaje) : void

Estados (patrón State)

«abstract» **EstadoReserva**
+ nombre() : String
+ activar(r : Reserva) : void
+ cancelar(r : Reserva) : void
+ vencer(r : Reserva) : void
+ finalizar(r : Reserva) : void

ReservaPendiente
+ activar(r : Reserva) : void
+ cancelar(r : Reserva) : void
+ vencer(r : Reserva) : void

ReservaFinalizada

ReservaActiva
+ finalizar(r : Reserva) : void

ReservaCancelada
ReservaVencida

«abstract» **EstadoPedido**
+ nombre() : String
+ avanzar(p : Pedido) : void
+ cancelar(p : Pedido, motivo : String) : void

PedidoConfirmado
+ avanzar(p : Pedido) : void
+ cancelar(p : Pedido, motivo : String) : void

PedidoListo
+ avanzar(p : Pedido) : void

PedidoEnPreparacion
+ avanzar(p : Pedido) : void

PedidoEntregado
PedidoCancelado

«abstract» **EstadoPago**
+ nombre() : String
+ aprobar(p : Pago) : void
+ rechazar(p : Pago) : void

PagoPendiente
+ aprobar(p : Pago) : void
+ rechazar(p : Pago) : void

PagoAprobado
PagoRechazado

Cada estado acepta solo sus transiciones válidas; las demás las rechaza la clase base. Se persiste por nombre() en la columna estado (DER sin cambios).

Enumeraciones

«enum» **EstadoCuenta**
- ABIERTA
- CERRADA

«enum» **ModoAsignacion**
- MANUAL
- AUTOMATICA

«enum» **MetodoPago**
- TARJETA
- BILLETERA_VIRTUAL

«enum» **TipoEvento**
- RESERVA_CREADA
- RESERVA_ACTIVADA
- RESERVA_CANCELADA
- RESERVA_VENCIDA
- PEDIDO_CONFIRMADO
- PEDIDO_CAMBIO_ESTADO
- PEDIDO_CANCELADO
- PAGO_REGISTRADO
- CUENTA_MESA_CERRADA
- PLATO_SIN_RECETA
- STOCK_INSUFICIENTE
- ALERTA_CREADA
- PLANIFICACION_CONFIRMADA
- VISITA_REGISTRADA
- PUNTOS_ACREDITADOS
- BENEFICIO_DESBLOQUEADO

«enum» **UnidadMedida**
- GRAMO
- KILOGRAMO
- MILILITRO
- LITRO
- UNIDAD
+ dimension() : Dimension
+ factorABase() : Decimal

«enum» **Dimension**
- MASA
- VOLUMEN
- CONTEO

«enum» **TipoMovimiento**
- INGRESO
- CONSUMO_PEDIDO
- DEVOLUCION_PEDIDO
- MERMA
- AJUSTE

«enum» **TipoMovimientoPuntos**
- ACREDITACION
- CANJE
- AJUSTE

«enum» **TipoDescuento**
- PORCENTAJE
- MONTO_FIJO

«enum» **Arquetipo**
- MOZO
- COCINERO
- OTRO

«enum» **FranjaServicio**
- ALMUERZO
- CENA

«enum» **EstadoPlanificacion**
- BORRADOR
- CONFIRMADA

«enum» **MetodoPronostico**
- PROMEDIO_MOVIL_PONDERADO
- NAIVE_ESTACIONAL
- REGRESION_ESTACIONAL

«enum» **TipoAlerta**
- PLATO_SIN_RECETA
- STOCK_INSUFICIENTE
- FALTANTE_STOCK_PREVISTO
- FALTANTE_PERSONAL
- CAPACIDAD_MESAS

«enum» **Severidad**
- INFO
- ADVERTENCIA
- CRITICA

Convenciones
Entidad de dominio · Servicio / tarea programada / builder
Interfaz (Strategy / Observer) · Objeto valor (no se persiste)
Enumeración (se usa como tipo de atributo; sin líneas para no saturar)
atributo privado − operación pública + operación de paquete
cursiva = abstracta · subrayado = estática